

Project Architecture

Monorepo Structure & Development Plan

ARCHITECTURE GOAL

The repository is organized as a simple monorepo so that frontend work, backend work, documentation, and testing remain clearly separated. This makes the project easier to maintain and extend.

REPOSITORY STRUCTURE

Folder / File	Purpose
client/	Frontend application
server/	Backend / API boundary
docs/	Audit and architecture documentation
test/	Automated and manual test assets
README.md	Project overview
package.json	Project configuration
.gitignore	Files excluded from Git

APPLICATION BOUNDARIES

- Client: User interface, pages, components, styles, and accessibility behavior.
- Server: API endpoints, business logic, and future data services.
- Docs: Audit reports, architecture notes, evidence, and project decisions.
- Test: Accessibility, functional, integration, and regression tests.

FIRST VERTICAL FEATURE SLICE

Open the target website → collect WAVE findings → run Lighthouse → perform keyboard checks → record evidence → document issues → recommend fixes → rerun tests after changes.

QUALITY PRINCIPLES

- Use semantic HTML and meaningful headings.
- Keep controls keyboard accessible with visible focus.
- Provide meaningful alternative text.
- Maintain sufficient color contrast.
- Keep frontend and backend responsibilities separate.
- Test important changes before release.

FUTURE EXPANSION

The structure can later support automated accessibility testing, Lighthouse CI, a small audit dashboard, issue tracking, API integration, and continuous integration through GitHub Actions.

CLIENT-READY OUTCOME

The architecture provides a clean foundation for demonstrating an accessibility-focused development process from audit evidence through implementation and verification.